

Пример минимального плана описания тестов

Должно быть проведено как минимум 3 вида тестирования.

Функциональное тестирование

Цель: проверка соответствия требованиям

Объект: варианты использования пользователя.

Метод: ручное тестирование системы как "черного ящика".

Техника: тестирование на основе формальной спецификации.

Приводятся тест-кейсы.

Нагрузочное тестирование

Цель: проверки выполнимости нефункционального требования – максимальное время ожидания ответа от сервера составляет 60 секунд (надежность).

Объект: API-маршруты автоматического создания чеков.

Метод: полу-автоматизированное тестирование системы как «серого ящика». Тестирование проводится с использованием инструмента Postman.

Техника: операционный профиль.

Приводятся данные для теста, скриншоты работы программы, при наличии - листинг кода.

Модульное тестирование

Цель: определение минимального тестируемого модуля (функция) и выявление ошибки в каждом отдельном модуле.

Объект: модуль создания чека

Метод: автоматизированное тестирование системы как белого ящика.

Техника: специализированное тестирование.

Приводятся unit-тесты.

Нефункциональное тестирование

Цель: поиск дефектов, не относящихся к поведению системы

Объект: страницы клиентской части приложения, а также запросы на сервер

Метод: ручное тестирование системы как «белого ящика»

Удобство использования

- Протестировать автоматическое формирование структуры пользовательского интерфейса в зависимости от роли
- Протестировать элементы пользовательского интерфейса на предмет единообразия форматирования текста и стилей
- Протестировать автоматическое заполнение значений известными данными.

Безопасность

- Протестировать систему на невозможность осуществлять те функции, которые не предусмотрены доступам и полномочиям пользователя.
- Протестировать наличие шифрования данных при передаче между различными частями системы должны быть зашифрованы.

Техника указывается в зависимости в соответствии с атрибутом качества.

Приводятся данные для тестирования и по итогам тестирования.

Пример плана тестирования

Объект: ...

Проводимые испытания по уровням тестирования

1 Модульное тестирование (Unit testing)

Цели: определение минимального тестируемого модуля (например, класс) и выявление ошибки в каждом отдельном модуле.

Техника: например, Специализированное тестирование или Объектно-ориентированное тестирование.

Действия: проверить правильность работы модуля (класса) «...».

2 Интеграционное тестирование (Integration testing)

Цели: проверка взаимодействия между программными модулями/классами (тестируются интерфейсы)

Техника: например, Специализированное тестирование или Объектно-ориентированное тестирование.

Действия: проверить правильности передачи данных между модулями/классами ... и ...

3 Системное тестирование (System testing)

Цели: тестирование системы в целом на её соответствие требованиям

На базе требований техника: например, Тестирование на основе формальной спецификации.

Действия: проверить все требования (test case), предъявляемые преподавателем.

На базе случаев использования техника: Функциональное и структурное.

Действия: выявление ошибок в работе функций, отображении информации на экране, поиск непредусмотренных сценариев действий пользователя (возможность/невозможность того или иного действия без предварительных шагов).

По технике управления качеством программного обеспечения

1 Статистический метод

Цели: выявление ошибок без запуска программного кода до этапа компиляции программы.

Техника: например, Тестирование, базирующееся на надежности инженерного процесса

Действия: проверить ошибки, используя встроенный в платформу разработки синтаксис-помощник.

2 Динамический метод:

Цели: выявление ошибок с компиляцией запускаемого файла.

Техника: например, Исследовательское тестирование

Действия: проверка кода программы непосредственно работой исполняемого файла.

По видам тестирования выделим следующие испытания

По объекту тестирования

1 Функциональное тестирование (functional testing)

Цели: проверка соответствия требованиям функциональности.

Техника: например, Функциональное и структурное или Тестирование на основе формальной спецификации.

Действия: проверить возможность/невозможность функции программы ...

2 Тестирование производительности (perfomance/stress testing)

Цели: проверка времени отклика программы в условиях прогнозируемой нормальной нагрузки.

Техника: например, Операционный профиль.

Действия: проверить время отклика наиболее критичных функций программы.

3 Тестирование стабильности (stability/load testing)

4 Тестирование удобства использования (usability testing)

5 Тестирование интерфейса пользователя (UI testing)

6 Тестирование безопасности (security testing)

7 Тестирование локализации (localization testing)

8 Тестирование совместимости (compartability testing)

По времени проведения тестирования

1 Альфа тестирование (alpha testing)

Цели: Проверка соответствия системы требованиям, описанным предъявляемым к системе.

Методы:

Тестирование при приёмке (smoke testing)

Тестирование новых функциональностей (new feature testing)

Регрессионное тестирование (regression testing)

Тестирование при сдаче (acceptance testing)

2 Бета тестирование (beta testing)

По степени автоматизации

1 Ручное тестирование (manual testing)

2 Автоматизированное тестирование (automated testing)

По учебному графику запланированы в следующем семестре.

3 Полуавтоматизированное тестирование (semiautomated testing)

По учебному графику запланированы в следующем семестре.

По признаку позитивности сценариев

1 Позитивное тестирование (positive testing)

2 Негативное тестирование (negative testing)

По степени подготовленности к тестированию

1 Тестирование по документации (formal testing)

2 Эд Хок (интуитивное) тестирование (ad hoc testing)

Примеры применения техник и оформления проводимых тестов

Эквивалентное разбиение

Проверка вводимого формата данных в полях ввода

Входное условие	Правильные классы эквивалентности	Неправильные классы эквивалентности
Объем: Числовые данные $(-\infty; 0,01]$; Числовые данные $[0,01; 9\,999,99]$; Числовые данные $(9\,999,99; +\infty)$	Числовые данные $[0,01; 9\,999,99]$	Числовые данные $(-\infty; 0,01]$; Числовые данные $(9\,999,99; +\infty)$
День: Числовые данные $(-\infty; 0]$; Числовые данные $[1; 31]$; Числовые данные $(32; +\infty)$	Числовые данные $[1; 31]$	Числовые данные $(-\infty; 0]$; Числовые данные $(32; +\infty)$

Предположение ошибок

- Попытка удаления в пустом справочнике
- Удаление объекта, который используется другими элементами

Список тестовых случаев

Номер TestCase	1
Функция	Удаление записи из справочника
Действие	Попытка удаления из пустого справочника
Ожидаемый результат	Блокирование кнопки «Удаления»
Предусловие	На главной форме выбрать пункты меню: «Справочники»- Название необходимого справочника. Справочник пуст
Шаги теста	1. Нажатие кнопки «Удалить»
Постусловие	
Результат	Тест пройден

Номер TestCase	2
Функция	Удаление записи из справочника
Действие	Удаление объекта, который используется другими элементами
Ожидаемый результат	Вывод сообщения об ошибке: «Удаление невозможно! Объект используется другими элементами.»
Предусловие	На главной форме выбрать пункты меню: «Справочники»- Название необходимого справочника. Заполненные объекты используются другими элементами
Шаги теста	1. Выделить строку, которую необходимо удалить 2. Нажать на кнопку «Удалить» на форме справочника
Постусловие	
Результат	Тест пройден

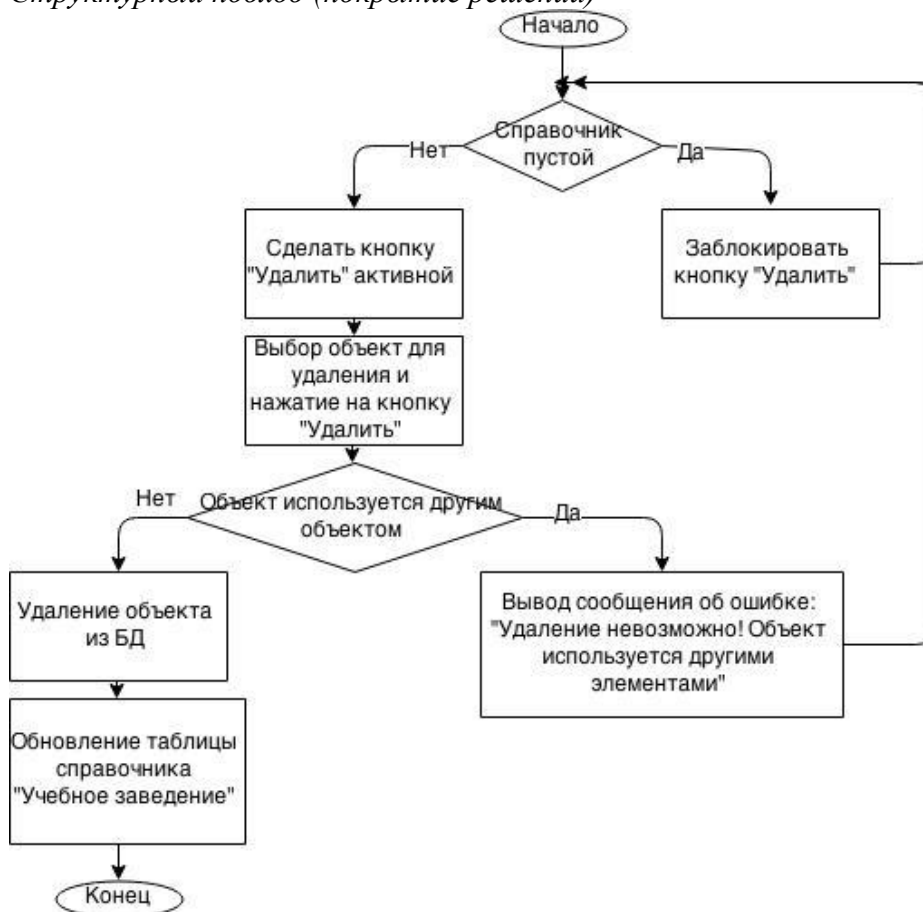
Таблица принятия решения

Условие	Значение «+» - да, «-» - нет	П1	П2	П3
При удалении справочник пустой	+, -	+	-	-
Удаление объекта, который используется другими элементами	+, -	-	-	+
Действия				
Блокирование кнопки «Удалить»		+	-	-
Вывод сообщения: «Удаление невозможно!» Объект используется другими элементами		-	-	+
Удаление данных из БД		-	+	-
Обновление таблицы справочника на форме		-	+	-

Структурное тестирование

Согласно спецификации, система должна разветвление сценариев для UC-04. Удаление данных из справочника.

Структурный подход (покрытие решений)



Функциональное тестирование

Согласно спецификации, функция Удаление данных из справочника должна содержать следующие подфункции:

- **UC-04.0; UC-04.1**

Согласно спецификации, в случае невозможности удаления записи из справочника должно срабатывать исключение

- **UC-04.0.E.0. Объект используется другими объектами, удаление невозможно**

Список тестовых случаев

Номер TestCase	1
Функция	Удаление записи из справочника
Действие	Удаление выделенной строки из таблицы справочника
Ожидаемый результат	Удаление объекта из БД и обновление таблицы справочника на форме
Предусловие	На главной форме выбрать пункты меню: «Справочники»-Название необходимого справочника. В справочнике имеется несколько записей
Шаги теста	1. Выделить строку, которую необходимо удалить 2. Нажать на кнопку «Удалить» на форме справочника
Постусловие	
Результат	Тест пройден

Номер TestCase	2
Функция	Удаление записи из справочника
Действие	Попытка удаления из пустого справочника
Ожидаемый результат	Блокирование кнопки «Удаления»
Предусловие	На главной форме выбрать пункты меню: «Справочники»- Название необходимого справочника. Справочник пуст
Шаги теста	1. Нажатие кнопки «Удалить»
Постусловие	
Результат	Тест пройден

Номер TestCase	3
Функция	Удаление записи из справочника
Действие	Удаление из пустого справочника одной единственной записи
Ожидаемый результат	Удаление объекта из БД, обновление таблицы справочника на форме, блокирование кнопки «Удалить»
Предусловие	На главной форме выбрать пункты меню: «Справочники»- Название необходимого справочника. Справочник имеет единственную запись
Шаги теста	1. Нажатие кнопки «Удалить»
Постусловие	
Результат	Тест пройден

Номер TestCase	4
Функция	Удаление записи из справочника
Действие	Удаление объекта, который используется другими элементами
Ожидаемый результат	Вывод сообщения об ошибке: «Удаление невозможно! Объект используется другими элементами.»
Предусловие	На главной форме выбрать пункты меню: «Справочники»- Название необходимого справочника. Заполненные объекты используются другими элементами
Шаги теста	1. Выделить строку, которую необходимо удалить 2. Нажать на кнопку «Удалить» на форме справочника
Постусловие	
Результат	Тест пройден

Тестирование было произведено посредством написания и проведение unit-тестов

1. Регистрация пользователя с некорректными данными

```
def test_signup_with_incorrect_credentials(self):
    """Тестирование регистрации пользователя с несоответствующими шаблону данными"""
    self.client.raise_request_exception = True
    # Пароль больше 12 символов
    resp = self.signup('Username7', 'name7', 0, 'Password638753893101')
    self.assertEqual(resp.status_code, 401)
    # Пароль меньше 6 символов
    resp = self.signup('Username41', 'name7', 0, 'Pass')
    self.assertEqual(resp.status_code, 401)
    # Логин больше 30 символов
    resp = self.signup('Username7sdfasefgsrfrhrcskidfhtasd', 'name7', 0, 'Password4')
    self.assertEqual(resp.status_code, 401)
    # Логин меньше 4 символов
    resp = self.signup('Use', 'name7', 0, 'Password4')
    self.assertEqual(resp.status_code, 401)
    # Имя больше 30 символов
    resp = self.signup('Username7sdfasefgsrfrhrcskidfhtasd', 'name7sdfasefgsrsefgsrfrhrcskidfhtasd', 0, 'Password4')
    self.assertEqual(resp.status_code, 401)
    # Имя меньше 4 символов
    resp = self.signup('Use', 'nam', 0, 'Password4')
    self.assertEqual(resp.status_code, 401)
```

Рисунок 1. Часть листинга unit-теста

Покрытие:

Unit-test затрагивает следующую функциональность:

- Добавление нового пользователя в систему с некорректными данными (User)

Результаты тестирования:

Тест пройден успешно, время выполнение теста составляет 1 ms.

✓ Test Results	1 ms
✓ ShowResults.test.tsx	1 ms
✓ ShowResults	1 ms
✓ check results	1 ms

Нагрузочное тестирование

Цель: проверка поведения работы системы на больших данных

Объект: функция получения масок объектов

Метод: тестирование системы как «белого ящика»

Для нагрузочного тестирования был реализован метод для считывания файлов из директории и определения масок с помощью модели.

Код метода:

```
def test(count):
    model = segmentation.ResUnet("test")
    model.get_model()

    list_imgs = glob.glob('data/images/*')
    start_time = datetime.now()
    count = int(count / len(list_imgs))

    for i in range(count):
        images = [cv.imread(file) for file in list_imgs]
        masks = [model.predict(img) for img in images]

    print(datetime.now() - start_time)
```

Рисунок 1. Листинг unit-теста

Результат тестирования представлен в таблице 1.

Таблица . Результат нагрузочного тестирования

Номер теста	Количество изображений	Время формирования
1	100	7 секунд
2	1 000	74 секунды
3	10 000	751 секунда

Тестирование удобства использования/ Тестирование пользовательского интерфейса UI

Объект тестирования: команды пользователя/экраны приложения

Ход тестирования: были приглашены независимые пользователи, которые опробовали функционал системы и составили оценку, опираясь на следующие критерии: интуитивность, скорость, отсутствие ошибок при работе, общее удовлетворение от работы с системой. Был проставлен рейтинг по каждому из пунктов, опираясь на шкалу от 0 до 10, где 0 – критерий не выполняется, 10 – критерий выполняется в полной мере.

Приводятся действия пользователя и результат работы – соответствующие экраны приложения.

Результаты тестирования:

Была составлена общая таблица оценок интерфейса пользователями.

Средняя оценка равна 8,5. Система соответствует указанным требованиям.

Таблица Оценки интерфейса пользователями.

Пользователь	Интуитивность	Скорость	Отсутствие ошибок при работе	Общее удовлетворение от работы	Результат
Tester_1	9	9	8	9	8,75
Tester_2	9	9	9	9	9
Tester_3	7	9	7	8	7,75
Tester_4	7	10	8	9	8,5
Tester_5	8	9	8	9	8,5
Tester_6	9	10	9	10	9,5

Рекомендации: стоит обратить внимание на проверку ошибок, описанные в рамках функционального тестирования, так как низкая оценка критерия «Отсутствие ошибок при работе».

Баг репорт:

При вызове команды /export сообщение отсылается 2 раза на почту.